

LumiBank

Relazione Tecnica

Sistema di gestione bancaria web-based

Autori	Daniele Coceancig, Alex Troilo, Giuseppe Guarano
Tecnologie	HTML5 · CSS3 · JavaScript (ES6+) · IndexedDB · EmailJS
Tipologia	Applicazione web single-page (SPA)
Anno	2026

Indice

1. Introduzione e contesto progettuale
2. Architettura generale del sistema
3. Struttura dei dati
 - 3.1 Entità Cliente
 - 3.2 Entità Conto Corrente
 - 3.3 Entità Prestito
 - 3.4 Entità Movimento
4. Moduli funzionali principali
 - 4.1 Autenticazione e gestione sessione
 - 4.2 Dashboard e statistiche
 - 4.3 Gestione conti
 - 4.4 Operazioni bancarie
 - 4.5 Gestione prestiti
5. Sistema di persistenza – IndexedDB e localStorage
6. Integrazione EmailJS
7. Pattern architetturali e scelte progettuali
8. Sicurezza
9. Conclusioni e sviluppi futuri

1. Introduzione e contesto progettuale

LumiBank è un'applicazione web di gestione bancaria sviluppata interamente con tecnologie front-end (HTML5, CSS3, JavaScript ES6+). Il sistema simula le funzionalità core di una banca digitale, consentendo agli operatori di filiale (staff) e ai clienti di operare attraverso un'interfaccia unica, responsive e moderna.

Il progetto nasce con l'obiettivo di dimostrare la fattibilità di un sistema bancario funzionale senza back-end dedicato, sfruttando le API di persistenza native del browser (IndexedDB e localStorage) e servizi cloud di terze parti per la notifica via email (EmailJS).

Il sistema è stato progettato e sviluppato da **Daniele Coceancig**, **Alex Troilo** e **Giuseppe Guarano**.

2. Architettura generale del sistema

LumiBank adotta un'architettura Single-Page Application (SPA): una sola pagina HTML gestisce l'intera navigazione tramite mostrare/nascondere sezioni via JavaScript, eliminando il reload della pagina e garantendo un'esperienza fluida.

Livello	Tecnologia	Responsabilità
Presentazione	HTML5 + CSS3	Struttura UI, layout responsive, tema visivo
Logica	JavaScript ES6+	Business logic, OOP, gestione eventi, routing
Persistenza	IndexedDB	Salvataggio conti correnti (strutturato, asincrono)
Persistenza	localStorage	Clienti, prestiti, sessione utente
Notifiche	EmailJS	Invio email automatiche (registrazione, operazioni)

La separazione tra i due sistemi di storage è motivata dalla natura dei dati: i conti contengono lo storico dei movimenti (dati strutturati e potenzialmente voluminosi, ideali per IndexedDB), mentre clienti e prestiti sono collezioni flat più semplici.

3. Struttura dei dati

Il modello dati è composto da quattro entità principali, implementate come oggetti JavaScript con classi ES6. Tutte le relazioni sono gestite tramite ID univoci generati con `Date.now()` o `crypto.randomUUID()`.

3.1 Entità Cliente

Rappresenta il correntista registrato nel sistema. Ogni cliente possiede credenziali proprie e può accedere all'area personale per visualizzare i propri conti e operazioni.

Campo	Tipo	Descrizione
id	String	Identificatore univoco (UUID)
nome / cognome	String	Dati anagrafici
email	String	Usata come username di accesso
passwordHash	String	Password codificata in Base64
codiceFiscale	String	Identificativo fiscale
registraToll	String	Data di registrazione

3.2 Entità Conto Corrente

Nucleo del sistema finanziario. Ogni conto appartiene a un cliente e può essere di tre tipologie con caratteristiche e costi differenti.

Campo	Tipo	Descrizione
id	String	ID univoco
iban	String	Codice IBAN generato automaticamente
tipoConto	String	Canone Fisso Senza Canone Deposito
saldo	Number	Saldo attuale in euro
clientId	String	FK al cliente proprietario
titolari	Array	Lista degli eventuali co-intestatari
movimenti	Array	Storico cronologico delle operazioni
commissioni	Number	Commissione per operazione (dipende dal tipo)

Tipo conto	Canone mensile	Commissione op.	Particolarità
Canone Fisso ■	Sì	€ 3,00	Accesso completo a tutti i servizi
Senza Canone ■	No	Variabile	Nessun canone, commissioni per op.
Deposito ■	No	No	Solo versamento e prelievo; interessi

3.3 Entità Prestito

Gestisce le richieste di finanziamento. Ogni prestito è collegato a un cliente e può trovarsi in uno di tre stati: Richiesto, Accettato o Respinto.

Campo	Tipo	Descrizione
id	String	ID univoco
clientId	String	FK al cliente richiedente

Campo	Tipo	Descrizione
tipoPrestito	String	Personale Mutuo Auto Altro
importo	Number	Importo richiesto in euro
rata	Number	Rata mensile calcolata
durata	Number	Durata in mesi
motivo	String	Motivazione dichiarata
statoPrestito	String	Richiesto Accettato Respinto

3.4 Entità Movimento

Ogni operazione eseguita su un conto genera un record Movimento allegato all'array movimenti del conto. I movimenti non vengono mai cancellati, garantendo la tracciabilità.

Campo	Tipo	Descrizione
tipo	String	Entrata Uscita
tipoOperazione	String	Bonifico Versamento Prelievo Interesse...
importo	Number	Importo in euro
causale	String	Descrizione dell'operazione
dataContabile	String	Data e ora dell'operazione
categoria	String	Categoria opzionale (spesa, stipendio...)

4. Moduli funzionali principali

4.1 Autenticazione e gestione sessione

Il sistema prevede due modalità di accesso selezionabili dalla schermata di login:

- **Modalità Staff:** accesso diretto (nessuna password) con piena visibilità su clienti, conti e prestiti. Lo staff può creare clienti, aprire conti, gestire prestiti e visualizzare tutti i movimenti.
- **Modalità Cliente:** accesso tramite email e password. Il cliente vede solo i propri conti e può eseguire operazioni bancarie autonomamente.

La password viene codificata tramite Base64 (funzione `hashPassword`) prima della memorizzazione. La sessione è gestita tramite variabili globali e `sessionStorage`.

4.2 Dashboard e statistiche

La dashboard è la schermata principale per lo staff e presenta in tempo reale tre indicatori chiave renderizzati dalla classe **StatsRenderer**:

- Numero totale di clienti registrati
- Numero totale di conti aperti
- Numero di richieste di prestito in attesa (stato: Richiesto)

4.3 Gestione conti

La sezione Conti (accessibile allo staff) mostra tutti i conti aperti in una griglia di card. Ogni card riporta tipo di conto, IBAN, titolari e saldo corrente. Cliccando una card si apre il modal di dettaglio con due tab: *Informazioni* (dati completi) e *Movimenti* (storico cronologico). Il rendering è delegato alla classe statica **CardRenderer**.

4.4 Operazioni bancarie

La sezione Operazioni (visibile ai clienti) permette di eseguire transazioni sul conto selezionato. Le operazioni disponibili variano in base al tipo di conto:

Operazione	Canone Fisso	Senza Canone	Deposito
Bonifico	✓	✓	x
Versamento	✓	✓	✓
Prelievo	✓	✓	✓
Pagamento	✓	✓	x
Maturazione int.	x	x	✓

Ogni operazione aggiorna il saldo del conto, aggiunge un record movimento e, se EmailJS è configurato, invia una notifica email al cliente.

4.5 Gestione prestiti

Lo staff può visualizzare tutte le richieste di prestito e aggiornarne lo stato (Accettato / Respinto). Il cliente può richiedere un prestito dalla propria area indicando tipo, importo, durata e motivazione.

La rata mensile viene calcolata automaticamente. All'accettazione o rifiuto, viene inviata una notifica email.

5. Sistema di persistenza – IndexedDB e localStorage

LumiBank adotta una strategia di persistenza ibrida, scegliendo lo strumento più adatto in base alla natura e al volume dei dati:

Storage	Dati memorizzati	Motivazione
localStorage	Clienti, prestiti	Strutture semplici, accesso sincrono rapido
IndexedDB	Conti correnti + movimenti	Dati strutturati, operazioni asincrone, capacità maggiore
sessionStorage	Sessione utente corrente	Dati volatili, cancellati alla chiusura del browser

La funzione `initDB()` inizializza il database IndexedDB con l'object store **conti** alla prima esecuzione. Le operazioni di lettura e scrittura sono incapsulate nelle funzioni `caricaContiIndexedDB()` e `salvaContiIndexedDB()`, entrambe asincrone (Promise-based). I dati sono automaticamente ripristinati ad ogni avvio dell'applicazione.

6. Integrazione EmailJS

LumiBank utilizza il servizio EmailJS per inviare notifiche email transazionali senza necessità di un server back-end. Sono definiti tre template distinti:

Evento	Template	Destinatario
Registrazione	EMAILJS_REGISTER_TEMPLATE	Cliente appena registrato
Approvazione prestito	EMAILJS_TEMPLATE_ID	Cliente (esito prestito)
Operazione bancaria	EMAILJS_OPERAZIONE_TEMPLATE	Cliente (dopo ogni operazione)

Il servizio è inizializzato all'avvio con la chiave pubblica del progetto. Tutte le chiamate sono gestite con `.then()/.catch()` per non bloccare l'interfaccia in caso di errore di rete.

7. Pattern architetturali e scelte progettuali

Class-based OOP

Le entità principali (Conto, Prestito) e i renderer (CardRenderer, StatsRenderer) sono implementati come classi ES6 con metodi statici, favorendo riuso e leggibilità.

Separation of Concerns

La struttura del codice separa chiaramente presentazione (HTML/CSS), logica di business (script.js) e persistenza (funzioni dedicate IndexedDB/localStorage).

No-Repetition via CardRenderer

La classe statica CardRenderer centralizza la generazione di HTML per card clienti, conti e movimenti, evitando duplicazione di codice nei vari punti di rendering.

Event-driven UI

L'interfaccia reagisce agli eventi utente (click, input) tramite handler JavaScript che aggiornano lo stato e ri-renderizzano i componenti interessati.

SPA Navigation

La navigazione tra sezioni (Dashboard, Conti, Clienti, Prestiti, Operazioni) è gestita dalla funzione `mostraSezione()` che alterna la visibilità dei blocchi DOM.

8. Sicurezza

Trattandosi di un'applicazione didattica client-side, le misure di sicurezza adottate sono quelle compatibili con l'architettura scelta:

- Le password vengono codificate in Base64 prima della memorizzazione (non in chiaro).
- La sessione staff non persiste oltre la sessione browser (`sessionStorage`).
- L'accesso alle funzioni amministrative è condizionato dalla variabile `sessionMode`.
- I dati sono isolati nel browser dell'utente (no condivisione tra dispositivi).

9. Conclusioni e sviluppi futuri

LumiBank dimostra come sia possibile realizzare un sistema bancario funzionale e visivamente curato utilizzando esclusivamente tecnologie web lato client. Il progetto copre i principali scenari operativi di una banca digitale: registrazione clienti, apertura conti, operazioni finanziarie, richiesta prestiti e notifiche email automatiche.

Possibili evoluzioni del progetto:

- Migrazione a un back-end Node.js/Python con database relazionale (PostgreSQL) per multi-utente reale.
- Implementazione di JWT per l'autenticazione sicura e gestione token di sessione.
- Crittografia end-to-end dei dati sensibili memorizzati in locale.
- Aggiunta di grafici e report analitici per lo staff (andamento saldi, volume operazioni).